

then extracts structured data from the raw HTML and returns it back to Scrapy.

To help the spiders extract data from web pages with minimal code required, Scrapy provides what it calls **selectors**. The CSS selector for instance let's you select data from an HTML page using the CSS selectors you'd use in stylesheets or in jQuery. Another option is to use XPATH, a powerful syntax for navigating XML data.

When you're done scraping, you can optionally structure your data even better by using Scrapy as an **Item** class. If you are scraping multiple kinds of data from one or more spiders this can be quite useful.

For cleaning data, Scrapy has Item Pipelines. This code is sent as an item, and after operating on its contents it can return a cleaned item or even simply discard the item if it is invalid or otherwise useless.

Setting up a Scrapy project

Now that we have some idea about how a Scrapy project works, let's create our own, and write our first Spider. Let's begin by first installing Scrapy.

Scrapy is written in Python, so you need Python installed on your system to continue. You can get Python from python.org and we recommend you download the latest release of Python 3. Scrapy can also work with Python 2 if you already have it installed.

After Python is installed, installing Scrapy can be done via:

```
pip install --user scrapy==1.3
```

This will install Scrapy 1.3 which is the latest as of this writing. The `--user` option will ensure that Scrapy and its dependencies are installed for your own user account and don't interfere with the rest of your system.

Installing Scrapy gives you access to the `scrapy` command that can be used to set up new Scrapy projects. Let's create a project called 'github' with the `startproject` command:

```
scrapy startproject github
```

This command creates the Scrapy project structure in a folder called github. We can run the `genspider` command in the project directory to create a new spider.

```
cd github
```

```
scrapy genspider trending github.com
```

This creates a spider called 'trending' that will scrape pages from the `github.com` domain. This will create a file called `trending.py` in the spiders directory of our project. There is nothing special about the `genspider` command; if you follow the format you can create this file manually.

The Scrapy Shell

Before writing any code, it's always best to inspect the code of website you want to scrape, to look at how content is structured and how the data you want is nested within the HTML.

The Scrapy shell is a powerful environment to test and play with a website before you begin coding your actual scraper. It lets you run the kinds of commands you would run in a spider live in a REPL so you can look before you leap.

Here is how you can open the GitHub trending page in the shell:

```
scrapy shell https://www.github.com/trending
```

Scrapy will fetch this webpage, store it in a variable called `response` and give you tools to play with that data in an Python REPL. If you run the above command, right now though, you will get an empty response because GitHub signals to bots that they are disallowed from accessing the website.

These instructions are present in a `robots.txt` file on the website root, and tell responsible crawlers how to crawl the website. GitHub doesn't expect people to be crawling their website since they already provide an API for all their needs. For this learning exercise, we will tell Scrapy to ignore the `robots.txt` file.

Quit the shell by typing `quit()` or pressing `Ctrl-d` and edit the `settings.py` file in the project directory. Here you will notice a line that says `ROBOTSTXT_OBEY = True` set this to `False`. While this file is open, have a look at the options available. There is a lot you can configure such as concurrent requests, delays, cookies, headers, throttling etc.

Now rerun the `scrapy shell` command as mentioned above and you will see that the response variable is no longer empty. Now we can use those selectors to get to the content inside the page using CSS:

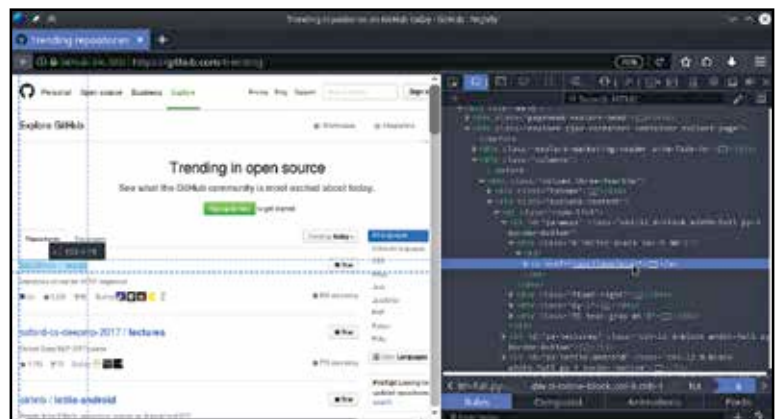
```
response.selector.css('h1').extract()
```

Here, we are using the CSS selector on the response and asking it to return all the h1 elements on the page (usually there is just one). The selector just makes a selection, and the `extract()` after that tells the selector to return the matched content as strings in a list. Here is what you should see:

```
['<h1>\n  <a class="pagehead-heading" href="/\n  explore">\n    Explore GitHub\n  </a>\n  </h1>']
```

As you can see, the h1 tag is inside it, now we know we need to go deeper. Instead of selecting `h1` we can select `h1>a` which selects an element directly inside an `h1` element. This will give us a tag, but what we really want is the title, the text that's actually inside the tag, not the tag itself.

While this is something you'd never need in regular CSS, Scrapy provides a syntax for that as well. Whenever you want the text content inside a tag, but not the tag itself, you can append `::text` at the end of your selector. If you want an attribute of the tag, you can use `::attr(...)`. For instance, to get



Exploring the structure of the GitHub trending repositories page to see how to write selectors for the bits we want.